ADS205 Tutorial 3 - Regular Expressions and the Pumping Lemma

September 23, 2026

1 ADS205 — Decision, Computation and Language

2 Tutorial 3 — Regular Expressions and the Pumping Lemma

Solutions are folded beneath each problem — click the arrow to expand them. Attempt the problem first.

2.1 Reference card

2.1.1 Regular expressions

Over an alphabet Σ, the base regular expressions are $\emptyset$, ε, and each symbol $a \in \Sigma$. If R and S are regular expressions, then so are

Expression	Language
$R \cup S$	$L(R) \cup L(S)$
RS	$L(R)L(S)$
R^*	$L(R)^*$

Operator precedence is: star first, then concatenation, then union.

2.1.2 Pumping lemma

If L is regular, then there exists $p \geq 1$ such that every $s \in L$ with $|s| \geq p$ can be written as $s = xyz$ where

$$|xy| \leq p, \qquad |y| > 0, \qquad xy^i z \in L \text{ for every } i \geq 0.$$

The lemma gives a **necessary** condition for regularity, not a sufficient one.

---

3 Part A — Regular expressions and finite automata

3.1 Problem 1 — What counts as evidence?

Answer each question and justify your answer.

1. If you cannot find a regular expression for a language, have you proved that it is not regular?
2. Is 0^n1^n a regular expression?
3. Is 0^*1^* a regular expression? Give one string in its language that is not of the form 0^n1^n.
4. If one long string can be pumped for every $i \geq 0$, does that prove the language is regular?
5. Why does the pumping lemma discuss strings of length at least p?

Solution — click to expand

1. **No.** Failure to find a description proves nothing. To prove regularity, exhibit a DFA, NFA, or regular expression. To prove non-regularity, use a valid impossibility argument such as the pumping lemma or closure properties.
2. **No.** The notation uses a variable n to force equal counts. Regular expressions use alphabet symbols, $\emptyset$, ε, and the operations union, concatenation, and star.
3. **Yes.** For example, `001111` belongs to $L(0^*1^*)$ but is not 0^n1^n for any n.
4. **No.** The pumping lemma concerns every sufficiently long string, with a suitable decomposition for each string. Passing one experiment is insufficient.
5. A DFA with p states must repeat a state while reading a sufficiently long accepted string. The repeated state creates the loop that can be pumped. A shorter run need not repeat a state.

3.2 Problem 2 — Convert $(0 \cup 1)^*01$ to an NFA

Construct an NFA equivalent to $(0 \cup 1)^*01$. State the role of every state and explain why the machine accepts exactly the strings ending in `01`.

Solution — click to expand

A compact equivalent NFA uses states q_0, q_1, q_2:

State	`0`	`1`
q_0 (start)	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\emptyset$	$\{q_2\}$
q_2 (accept)	$\emptyset$	$\emptyset$

State q_0 reads any prefix. On a `0`, the NFA may guess that this symbol begins the final `01` and move to q_1. The next symbol must be `1`, which reaches q_2. Because q_2 has no outgoing transitions, the successful `01` must occur at the end of the input.

3.3 Problem 3 — Convert a DFA to a regular expression

Consider the DFA below.

State	`0`	`1`
q_0 (start)	q_0	q_1
q_1	q_0	q_2
q_2 (accept)	q_2	q_2

1. Describe its language.

2. Use state elimination, or reason directly from the state meanings, to obtain a regular expression.
3. Explain why different state-elimination orders may produce different-looking answers.

Solution — click to expand

1. The DFA recognizes all binary strings containing `11` as a substring.
2. A regular expression is

$$(0 \cup 10)^*11(0 \cup 1)^*.$$

3. State elimination preserves the recognized language, but algebraically equivalent expressions can have different syntax. Elimination order changes the intermediate expressions and often the size of the final expression.

---

4 Part B — The pumping lemma

4.1 Problem 4 — Put the quantifiers in the correct order

Arrange the following actions into a valid pumping-lemma proof that a language L is not regular.

- Choose a string $s \in L$ depending on p.
- Choose a pumping exponent i that may depend on the decomposition.
- Assume that L is regular.
- Consider every decomposition $s = xyz$ satisfying $|xy| \leq p$ and $|y| > 0$.
- Obtain the pumping length p.
- Show that $xy^iz \notin L$, giving a contradiction.

Who controls p, s, the decomposition, and i in the usual pumping-lemma game?

Solution — click to expand

The correct order is:

1. Assume that L is regular.
2. Obtain the pumping length p promised by the lemma.
3. Choose a string $s \in L$ with $|s| \geq p$, usually depending on p.
4. Consider an arbitrary valid decomposition $s = xyz$.
5. Choose an exponent $i \geq 0$, possibly after seeing the decomposition.
6. Show that $xy^iz \notin L$, contradicting the lemma.

The lemma supplies p. The prover chooses s. An adversary chooses any valid decomposition. The prover then chooses i. A proof that handles only one convenient split is incomplete.

4.2 Problem 5 — Debug two pumping-lemma proofs

Let $L = \{0^n1^n \mid n \geq 0\}$. Both arguments below claim to prove that L is not regular. For each argument: (i) judge whether the proof is correct, (ii) if not, identify the invalid step, explain why it is invalid, and repair the proof.

Argument A

- Assume L is regular, and let p be its pumping length.
- Choose $s = 0^p 1^{p+1}$. Since $|s| \geq p$, write $s = xyz$ with $|xy| \leq p$ and $|y| > 0$.
- Then $y = 0^t$ for some $t \geq 1$.
- Pump down: $xz = 0^{p-t}1^{p+1} \notin L$.
- Therefore L is not regular.

Argument B

- Assume L is regular, and let p be its pumping length.
- Choose $s = 0^p 1^p$.
- Take the split $x = \varepsilon$, $y = 0$, and $z = 0^{p-1}1^p$. It satisfies $|xy| \leq p$ and $|y| > 0$.
- Pump down: $xz = 0^{p-1}1^p \notin L$.
- Therefore L is not regular.

Solution — click to expand

In **A**, the chosen string $s = 0^p 1^{p+1}$ is **not in** L. The pumping lemma promises a suitable decomposition only for strings in L of length at least p; the length condition alone is insufficient.

In **B**, the chosen split is valid, but examining **one** split cannot yield a contradiction. If L were regular, the lemma would guarantee at least one split that pumps for every exponent. To refute that claim, the proof must defeat **every** valid split.

Both arguments can be repaired as follows. Choose $s = 0^p 1^p \in L$, with $|s| = 2p \geq p$. Now let $s = xyz$ be an **arbitrary** decomposition satisfying $|xy| \leq p$ and $|y| > 0$. Because the first p symbols are all 0, necessarily $y = 0^t$ for some $t \geq 1$. Choosing exponent $i = 0$ gives

$$xy^0z = xz = 0^{p-t}1^p \notin L.$$

This works for every valid decomposition, contradicting the pumping lemma. Thus L is not regular.

4.3 Problem 6 — Similar definitions, different languages

For some $k \geq 1$, define

$$B = \{1^k w \mid w \in \{0,1\}^* \text{ contains at least } k \text{ ones}\},$$

$$C = \{1^k w \mid w \in \{0,1\}^* \text{ contains at most } k \text{ ones}\}.$$

1. Find a simple regular expression for B.
2. Give a pumping-lemma proof that C is not regular.

Solution — click to expand

For B, $k \geq 1$, and suppose we have a string $s \in B$, so $s = 1^k w$. Then,

$$s = 11^{k-1}w = 1(1^{k-1}w),$$

let $v = 1^{k-1}w$,, then we have $s = 1v$, which is a string in B with $k = 1$.

In other words, no matter what value k is, their corresponding strings are in the language B when $k = 1$.

Then let $k = 1$, we have \$B={1w|w has least 1 one.}, so the corresponding regualr expression is

$$B = 1(0 \cup 1)^* 1(0 \cup 1)^*.$$

Assume C is regular and let p be its pumping length.

Choose $s = 1^p 01^p$. It belongs to C by taking $k = p$.

For every decomposition $s = xyz$ with $|xy| \leq p$ and $|y| > 0$, we have $y = 1^t$ for some $t \geq 1$.

Pump down. The string $xz = 1^{p-t}01^p$ has an initial run of only $p - t$ ones, while the remaining suffix contains at least p ones, i.e., in the pattern, the k is less than or equal to $p - t$.

For xz to be inside C, we need to find a k that makes $k > p$, but $p > p-t$, so there is acontradiction, i.e., $xz \notin C$. Hence, C is not regular.

4.4 Optional Tock checks

Tock uses `|` for union, spaces for concatenation, `*` for Kleene star, and `&` for epsilon. It can turn a regular expression into an NFA and run that NFA. Finite tests illustrate a construction but do not prove a pumping-lemma claim.

[ ]:
```
# Install once in a fresh environment, then import Tock.
# %pip install -q tock
from tock import *

# B from Problem 6: strings that start with 1 and contain a later 1.
B_regexp = RegularExpression.from_str("1 (0 | 1)* 1 (0 | 1)*")
B_nfa = from_regexp(B_regexp)
display(B_regexp)
display(to_graph(B_nfa))

for word in ["&", "1", "1 0", "1 1", "1 0 1", "1 0 0 1 0", "1 1 1 0"]:
    graph = run(B_nfa, word)
    accepted = any(attrs.get("accept", False) for attrs in graph.nodes.values())
    print(f"{word!r:16} in B? {accepted}")
```

4.5 What to take away

- A regular expression, DFA, or NFA can prove that a language is regular. Failing to find one cannot prove non-regularity.
- Kleene's theorem connects regular expressions and finite automata in both directions. Thompson's construction handles regex to NFA; state elimination handles automata to regex.
- Pumping-lemma proofs depend on the quantifier order. Choose the witness after seeing p, handle every valid split, and choose the pumping exponent last.

- The witness string should expose the dependency that finite memory cannot maintain.